Sortir d’un ETL visuel : modèles d’exécution, traduction et validation d’équivalence des pipelines décisionnels

ZIDOUR Idir — Master 2 DS4SC, Université de Tours

Tuteur universitaire : DOUMARD Emmanuel — Entreprise d’accueil : Adiwatt

Rapport de phase bibliographique — S9

Résumé

Beaucoup d’organisations de taille moyenne intercalent, entre leurs bases de gestion et leur outil de restitution, un outil de préparation de données à flux visuel. Nous étudions ici la question de sa suppression : une chaîne de préparation bâtie dans un tel outil peut-elle être reconstruite à l’intérieur du moteur de transformation de l’outil de restitution, sans perte d’équivalence, et à quel coût ? Notre terrain est un pipeline industriel de 193 nœuds, exécuté quatre fois par jour, qui produit douze tables consommées par un unique rapport. Nous organisons la revue de littérature selon quatre propriétés transverses plutôt que par domaine de recherche : le lieu et le moment du calcul, les garanties du modèle d’exécution, l’établissement de la correspondance entre deux pipelines, et la preuve que la migration n’a rien changé. Il en ressort que chacune de ces propriétés est documentée isolément, mais qu’aucun travail ne traite la traduction d’un pipeline de flux visuel vers un langage de requêtes, ni l’équivalence d’une chaîne de traitement complète. Ce vide délimite la phase de réalisation : conduire la migration, et la valider par test différentiel contre les sorties de la chaîne d’origine, archivées avant son démantèlement.

1. Introduction

Un système décisionnel met en relation les bases de données des applications de gestion et les tableaux de bord que consultent les décideurs. Entre les deux, il faut nettoyer les données, les recouper et les agréger. Dans les grandes organisations, ce travail revient à une plateforme d’intégration administrée par une équipe dédiée. Dans les organisations de taille moyenne, il est le plus souvent confié à un outil de préparation à flux visuel, installé sur un poste et maintenu par une ou deux personnes. L’usage de cet outil s’étend ensuite par accumulation, au fil des besoins, et c’est cette trajectoire que nous étudions.

Le cas qui nous occupe en est un exemple représentatif. Une première base de données reçoit, sans retraitement, les extractions du progiciel de gestion d’une entreprise industrielle. Un pipeline de 193 nœuds et 219 connexions les lit intégralement, les nettoie, les recoupe, puis écrit douze tables dans une seconde base dite propre. Un rapport décisionnel de vingt-huit tables et d’une centaine de mesures est l’unique lecteur de cette seconde base. Une tâche planifiée du système d’exploitation enchaîne l’ensemble quatre fois par jour sur une machine dédiée, en moins de cinq minutes. Le dispositif fonctionne, ce qui rend d’ailleurs sa critique moins immédiate à formuler.

L’outil de flux employé est gratuit et la chaîne est rapide. La motivation de sa suppression n’est donc ni le coût d’une licence, ni la durée d’exécution, qui sont les deux arguments les plus souvent avancés dans la littérature professionnelle. Les motifs réels tiennent à trois points. D’abord, l’exécution est de type tout ou rien et ne sait pas reprendre : une valeur qu’un nœud ne parvient pas à traiter, à l’écriture en base ou dans une jointure, interrompt la totalité du traitement, y compris les branches qui n’ont aucun rapport avec elle, et le rapport reste sur les données de la veille jusqu’à ce qu’une exécution aboutisse. Ensuite, le dispositif impose de maintenir une machine dédiée, une base intermédiaire qui n’a pas d’autre usage, et deux paradigmes de transformation au lieu d’un. Enfin, la logique métier (quelles lignes sont exclues, comment une affaire est rattachée à un mois, ce que l’on appelle une livraison) n’existe que dans le graphe, sous un format de sérialisation propre à l’outil dont la sémantique est fixée par l’implémentation plutôt que par une spécification. Elle n’est donc pas documentée : elle est seulement exécutée.

De là vient la question qui structure notre travail : une chaîne de préparation bâtie dans un outil de flux visuel peut-elle être reconstruite à l’intérieur du moteur de transformation de l’outil de restitution,

sans perte d’équivalence, et à quel coût ? Chacun de ces termes demande à être précisé. Reconstruire n’est pas traduire, puisqu’il n’existe aucune règle de correspondance établie entre les deux langages en présence. Sans perte d’équivalence suppose d’avoir défini au préalable ce que l’on compare et avec quelle tolérance. À quel coût engage une mesure et non une préférence ; et comme la chaîne actuelle est déjà rapide, nous n’attendons pas que le gain porte d’abord sur la vitesse.

La difficulté ne vient pas des opérations elles-mêmes, qui sont élémentaires prises une à une, mais de trois propriétés du problème. La sémantique du pipeline d’origine n’est donnée que par son exécution, ce qui fait de sa compréhension un travail à part entière et non une formalité préalable. Les deux moteurs ne promettent pas les mêmes choses sur l’ordre des lignes, sur le sort des valeurs invalides et sur le lieu du calcul, de sorte qu’une traduction fidèle nœud à nœud ne garantit pas une sortie identique. Enfin, la référence à laquelle comparer le résultat, que le test logiciel appelle un oracle et que nous définissons en section 2.4, est la chaîne d’origine elle-même, qui a vocation à disparaître.

Le présent rapport constitue la phase bibliographique du projet. Il n’a pas pour objet de conduire la migration mais de l’instruire, et nous lui assignons trois objectifs. Nous voulons d’abord situer le problème, c’est-à-dire établir que les propriétés en jeu sont des objets d’étude constitués et non des préoccupations locales à une entreprise. Nous cherchons ensuite à nous outiller, en identifiant dans ces travaux les notions et les méthodes directement réutilisables pour la phase de réalisation. Nous voulons enfin délimiter le sujet, en disant ce que la littérature ne couvre pas, afin de pouvoir énoncer la contribution de la phase suivante par différence plutôt que par affirmation.

Notre revue s’appuie sur un corpus constitué dans les archives ouvertes et réduit à quinze travaux selon des critères que nous explicitons en section 3.1. Nous l’organisons non par domaine de recherche mais selon quatre propriétés transverses. La section 2 fixe le vocabulaire employé : la chaîne décisionnelle, les deux manières d’écrire une préparation de données, la délégation du calcul à la source, l’équivalence et l’oracle. La section 3 présente la revue selon les quatre axes annoncés. La section 4 récapitule ce que la littérature nous fournit et ce qu’elle laisse ouvert, et en déduit la démarche de la phase de réalisation.

2. Préliminaires

Nous fixons ici le vocabulaire employé dans la suite. Cette section s’adresse à un lecteur informaticien qui ne pratique pas nécessairement l’informatique décisionnelle ; le spécialiste peut la sauter.

2.1. La chaîne décisionnelle

Un système décisionnel relie des bases opérationnelles, celles que les applications de gestion alimentent au fil de l’eau, à des indicateurs consultés par des utilisateurs métier. Entre les deux s’intercalent trois étages : la préparation des données, leur modélisation, puis leur restitution.

La préparation est traditionnellement décrite par l’acronyme ETL, pour extraction, transformation et chargement. Un outil tiers lit les tables sources, applique une succession de transformations, puis écrit le résultat dans une base intermédiaire dite propre. L’approche concurrente, dite ELT, inverse les deux dernières étapes et transforme les données là où elles reposent. La différence ne porte pas seulement sur l’ordre des opérations, mais sur le lieu où le calcul s’exécute, question qui structurera l’essentiel de cette revue.

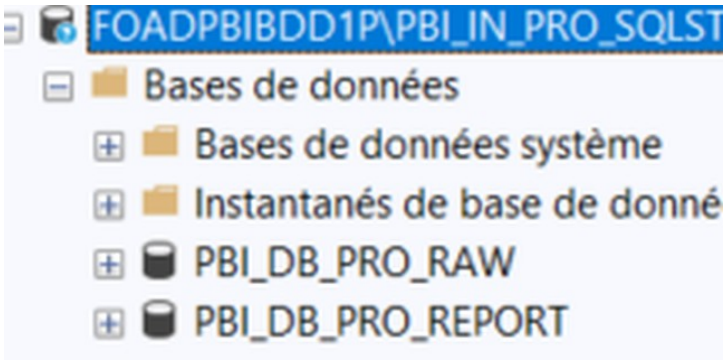


Figure 1 — Les deux bases du dispositif étudié. La base brute est alimentée par le progiciel de gestion ; la base intermédiaire est produite par l'outil ETL et consommée par le rapport. C'est cette seconde base que la migration vise à supprimer.

L'architecture que nous étudions relève du premier cas. Deux bases coexistent sur le même serveur : l'une reçoit les extractions du progiciel de gestion sans retraitement, l'autre ne contient que des tables produites par l'outil ETL à destination du rapport (figure 1). Cette base intermédiaire n'a pas d'autre usage et pas d'autre lecteur ; sa seule raison d'être est de stocker le résultat d'un traitement qui ne sait pas s'exécuter ailleurs.

L'ensemble n'est pas piloté par un ordonnanceur de traitements, mais par une tâche planifiée du système d'exploitation, sur un poste dédié (figure 2). Cette tâche enchaîne quatre fois par jour l'alimentation de la base brute, l'exécution du traitement et la publication du rapport. Elle journalise ses exécutions et notifie les échecs par courriel, mais elle ne sait pas reprendre : un traitement interrompu doit être relancé depuis son début, et rien n'est produit entre-temps.

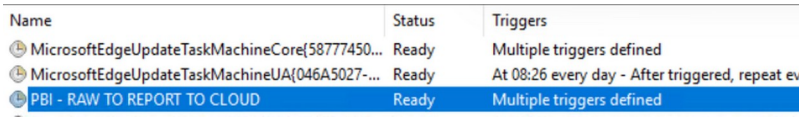


Figure 2 — L'orchestration de la chaîne : une tâche planifiée du système d'exploitation, déclenchée quatre fois par jour, qui enchaîne l'alimentation de la base brute, le traitement et la publication du rapport.

La modélisation organise ensuite les données en un schéma interrogeable, le plus souvent en étoile : des tables de faits, qui portent les événements mesurables, reliées à des tables de dimensions, qui portent les axes d'analyse. Les indicateurs sont exprimés sous forme de mesures, c'est-à-dire d'expressions agrégées évaluées à la demande selon le contexte de filtrage de l'affichage.

Nous appellerons pipeline la succession ordonnée de transformations menant d'une table source à une table exploitable, et opérateur chacune de ces transformations. Nous dirons qu'un résultat intermédiaire est matérialisé lorsqu'il est effectivement calculé et stocké, par opposition à un résultat qui n'existe que comme description d'un calcul à effectuer.

2.2. Deux manières d'écrire une préparation de données

La première consiste à composer un graphe orienté acyclique de nœuds. Chaque nœud est un opérateur paramétré par une interface graphique, et chaque arête transporte une table dont le schéma est connu. L'exécution progresse nœud par nœud, et la sortie de chacun est matérialisée avant que le suivant ne commence. L'outil que nous étudions relève de cette famille.

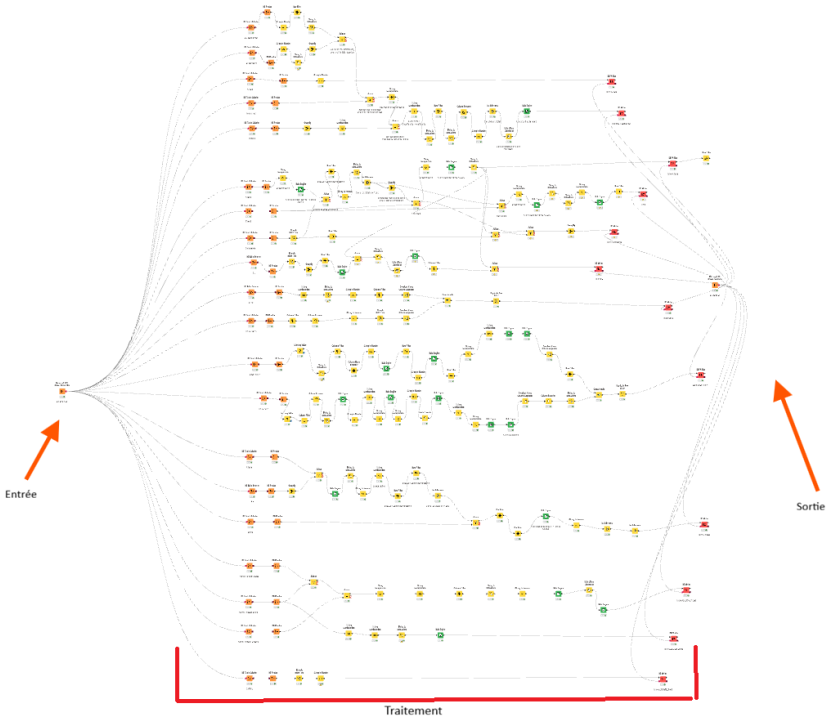


Figure 3 — Le pipeline étudié à l'ouverture de l'outil : 193 nœuds et 219 connexions, répartis en une vingtaine de branches parallèles. Les deux connecteurs d'entrée sont à gauche, les douze écritures de sortie à droite. Les annotations Entrée, Traitement et Sortie ont été ajoutées pour cette figure ; elles n'existent pas dans le fichier.

La figure 3 montre ce que devient un tel graphe après plusieurs années d'accumulation. La représentation visuelle, qui rend un petit flux immédiatement lisible, cesse de remplir cette fonction passé un certain volume. La documentation du traitement se confond alors avec le traitement lui-même, et disparaît avec lui.

La seconde manière consiste à écrire une suite d'expressions nommées dans un langage de transformation dédié. Les valeurs y sont immuables : une étape ne modifie pas la précédente, elle en dérive une nouvelle. Surtout, l'évaluation est paresseuse, c'est-à-dire que le moteur construit d'abord une description complète du calcul, puis décide comment l'exécuter. Cette différence, en apparence technique, a une conséquence directe : seul le second modèle autorise le moteur à réorganiser le travail avant de le faire.

Une troisième différence ne se révèle qu'en cas d'anomalie, et elle constitue l'un des motifs réels de la migration que nous étudions. Dans un flux de nœuds, l'erreur est un événement de contrôle : le nœud qui ne parvient pas à traiter une valeur échoue, et l'exécution s'arrête là où elle en est. Une seule ligne que l'un des nœuds ne sait pas traiter suffit à faire échouer la totalité du traitement, branches sans rapport comprises. L'échec est bien détecté, puisque la tâche journalise et alerte, mais il n'est pas rattrapé : le rapport reste sur les données de la veille jusqu'à ce qu'une exécution aboutisse. Dans un langage de transformation à valeurs immuables, l'erreur est au contraire une valeur comme une autre : la conversion qui échoue produit une cellule en erreur, la table continue d'exister et le traitement se poursuit. La différence est donc de nature, et elle décide de ce qui se passe le matin où une saisie inhabituelle remonte du progiciel de gestion.

2.3. Déléguer le calcul à la source

Lorsque la source de données sait elle-même exécuter une opération, comme filtrer, joindre ou agréger, un moteur à évaluation paresseuse peut traduire cette opération dans le langage de la source et la lui confier. Le résultat seul transite sur le réseau, et le volume intermédiaire ne quitte jamais le serveur de données. Les éditeurs désignent ce mécanisme par des termes propriétaires ; la littérature scientifique parle d'optimisation fondée sur les capacités de la source, ou de délégation de prédicats. Nous

emploierons ici le terme de délégation, en signalant les termes commerciaux lorsque nous citerons une documentation.

La délégation est partielle par nature : dès qu'une étape emploie une construction que la source ne sait pas exprimer, cette étape et toutes les suivantes doivent être évaluées localement. Le point où la chaîne se rompt détermine donc à lui seul le volume rapatrié.

2.4. Équivalence, tolérance et oracle

Nous dirons que deux pipelines sont équivalents si, pour les mêmes données d'entrée, ils produisent la même sortie. Cette définition apparemment triviale cache trois difficultés que la revue devra traiter. D'abord, l'égalité de deux tables peut s'entendre au sens des ensembles ou des multi-ensembles, selon que l'on tient compte ou non des doublons, et les tables réelles en contiennent. Ensuite, l'ordre des lignes n'est en général pas garanti, ce qui rend ambigu tout opérateur qui en dépend. Enfin, l'arithmétique à virgule flottante n'est pas associative : deux sommes des mêmes valeurs, calculées dans un ordre différent, peuvent différer de quelques unités du dernier chiffre significatif. Une égalité stricte est donc un critère inapplicable, et la question du seuil de tolérance se pose dès que l'on veut vérifier une migration.

Encore faut-il disposer de quelque chose à quoi comparer. Le test logiciel appelle oracle ce qui permet de trancher si le résultat observé d'une exécution est correct : une sortie de référence, une propriété que le résultat doit vérifier, ou une seconde implémentation réputée juste. La difficulté récurrente de ce domaine tient à ce que l'oracle manque le plus souvent, puisque l'on sait exécuter le programme mais non dire ce qu'il aurait dû produire. Une migration se trouve dans la situation inverse, et donc privilégiée : la chaîne à remplacer fonctionne encore, et ses sorties fournissent la référence exacte à laquelle confronter celles du pipeline reconstruit. Comparer ainsi deux implémentations d'un même traitement pour faire apparaître leurs divergences porte un nom, le test différentiel.

Cet avantage est temporaire, et c'est ce qui en fait une contrainte de calendrier. L'oracle est ici un système vivant : ses sorties changent à chaque exécution, puisque les données amont changent. Nous devrons donc le figer, c'est-à-dire archiver un export daté des tables produites assorti des paramètres d'exécution, avant toute modification ou tout démantèlement de la chaîne d'origine. Sans cette archive, la validation perd sa référence et l'équivalence devient indémontrable. C'est en ce sens que nous parlerons par la suite d'oracle figé.

3. Revue de littérature

3.1. Périmètre de l'étude

La question que nous posons, reconstruire une chaîne de préparation bâtie dans un outil de flux visuel à l'intérieur d'un autre moteur sans perte d'équivalence, mobilise plusieurs domaines de recherche qui se sont développés séparément. Les langages de programmation par flux de données fournissent le cadre théorique du premier outil. L'optimisation de requêtes réparties et la modélisation des processus ETL éclairent le placement du calcul. La rétro-ingénierie et la compréhension de programmes décrivent la reconstruction d'une sémantique perdue. La traduction automatique de code et l'ingénierie dirigée par les modèles traitent du portage lui-même. La vérification d'équivalence de requêtes, le test des systèmes orientés données et les travaux sur la reproductibilité fournissent enfin les instruments de la validation.

Nous laissons délibérément de côté quatre domaines voisins. Le traitement distribué à très grande échelle, d'abord : les volumes en jeu se comptent en dizaines de milliers de lignes, et les problématiques de partitionnement ou de tolérance aux pannes ne s'y appliquent pas. Le traitement en flux, ensuite : la chaîne étudiée est un traitement par lots, et les travaux sur le placement d'opérateurs en environnement réparti répondent à une contrainte de latence étrangère à notre sujet. La vérification formelle de programmes généralistes, également : nous retenons la vérification d'équivalence appliquée aux requêtes, mais non la preuve de programme au sens large, hors de portée de notre calendrier. L'apprentissage automatique appliqué aux données métier, enfin, qui relève de l'exploitation des indicateurs et non de la chaîne qui les produit.

Un travail nous sert de toile de fond pour la motivation plutôt que pour l'argumentation technique. Sculley et al. (2015) ont nommé les formes de dette que contracte tout système de traitement de données assemblé par accumulation : code de liaison, enchevêtrement de pipelines, dette de configuration. Ils montrent que le coût s'en paie en maintenance et non en développement. La dépendance à un outil unique en est un cas particulier, qui tient non au prix des licences mais à l'absence de représentation portable de la logique métier.

Plutôt que de présenter ces travaux domaine par domaine, ce qui reviendrait à juxtaposer des résumés, nous les organisons selon quatre propriétés transverses. Chacune est formulée comme une question que l'on peut poser à n'importe quelle approche : où et quand le calcul s'exécute ; ce que le modèle d'exécution garantit ; comment établir la correspondance entre deux pipelines ; comment prouver que la migration n'a rien changé. Un même travail peut donc apparaître sous plusieurs axes, éclairé à chaque fois sous un angle différent.

Les travaux que nous retenons proviennent d'un corpus initial d'une quarantaine de références, constitué par recherche par mots-clés sur les archives ouvertes et dans les actes librement accessibles des principales conférences du domaine. Deux critères ont guidé la réduction à quinze. Le premier est l'accès ouvert : l'établissement ne donne pas accès aux bibliothèques numériques de l'ACM, de l'IEEE et de Springer, ce qui écarte notamment la ligne de travaux consacrée au test des processus ETL, dont nous avons identifié les titres mais dont les textes nous sont inaccessibles. Le second est la contribution effective à l'un des quatre axes, à l'exclusion des travaux qui n'auraient fait qu'illustrer un point déjà établi. Les documentations d'éditeurs et les outils, nécessaires mais non évalués par les pairs, sont listés en annexe.

3.2. Premier axe : où et quand le calcul s'exécute

L'idée de confier une partie du travail à la source est ancienne. Haas et al. (1997) la formalisent : dans un système interrogeant des sources hétérogènes, l'optimiseur ne doit engendrer que les plans que les capacités déclarées de chaque source savent exécuter. La délégation n'est donc pas une astuce d'implémentation mais une contrainte intégrée à l'optimisation. Nous y voyons le rattachement académique d'un mécanisme que les éditeurs d'outils décisionnels décrivent aujourd'hui sous un vocabulaire propriétaire, sans jamais renvoyer à cette filiation.

Simitsis et al. (2005) déplacent la question à l'intérieur même du flux ETL. Ils le modélisent comme un état dans un espace d'états, et l'optimisent par des transformations (échange, factorisation, fusion d'opérateurs) dont ils démontrent qu'elles préservent la sémantique. Remonter un filtre en amont d'une jointure n'est alors plus une bonne pratique empirique mais une réécriture justifiée. Ce cadre nous est précieux : il fournit le langage permettant d'affirmer qu'une réorganisation du pipeline migré ne change pas son résultat.

Les travaux récents étendent la délégation au-delà du relationnel. Yan et al. (2023) délèguent des prédicats à travers des opérateurs non relationnels et des fonctions définies par l'utilisateur, en vérifiant formellement que la sortie du pipeline demeure inchangée. Cette contribution occupe une position charnière dans notre revue, car elle est la seule à traiter conjointement le déplacement du calcul et la preuve que ce déplacement est légitime. Elle montre aussi les limites de l'exercice : la délégation cesse d'être rentable dès que l'opérateur n'est que partiellement délégable, et le placement du calcul reste donc un arbitrage plutôt qu'une règle.

La seconde propriété de cet axe est temporelle. Johnston et al. (2004) décrivent les langages par flux de données comme des graphes où l'exécution est pilotée par la disponibilité de la donnée : chaque nœud produit un résultat complet avant que le suivant ne commence, et ce résultat est matérialisé. Aucune vue d'ensemble du calcul n'existe donc avant son exécution, et rien ne peut être réorganisé. Armbrust et al. (2015) donnent l'illustration inverse, avec une interface à évaluation différée doublée d'un optimiseur à règles qui repousse filtres et projections jusque dans la source. La paresse n'est pas ici une commodité d'écriture, c'est la condition de possibilité de la délégation étudiée plus haut.

Nous relevons pour finir un décalage entre la littérature et la pratique. La littérature traite le placement du calcul comme un problème d'optimisation automatique, résolu par le moteur. Dans les outils décisionnels que nous observons, il reste une décision de conception prise par un humain, souvent

implicite et sans instrumentation permettant d'en mesurer les conséquences. Cet écart constitue l'une des motivations de notre travail.

3.3. Deuxième axe : ce que le modèle d'exécution garantit

Un pipeline ne se caractérise pas seulement par ce qu'il calcule, mais par ce qu'il promet. L'ordre des lignes est-il stable d'une exécution à l'autre ? Le même jeu d'entrée produit-il toujours la même sortie ? Ces questions paraissent secondaires jusqu'au moment où un opérateur en dépend.

Shi et al. (2016) ont formalisé le problème dans un tout autre contexte, celui des bibliothèques logicielles. Ils nomment ainsi le code qui suppose déterministe une spécification qui ne l'est pas, et proposent de le détecter en explorant systématiquement les ordres d'itération licites. La transposition au domaine des données est immédiate. Un opérateur d'agrégation qui retient la première valeur rencontrée pour un groupe n'a de sens que si l'ordre de lecture est garanti ; or ni un flux de nœuds ni un langage de transformation ne le garantissent, et aucun des deux ne le documente. Le résultat est stable tant que les données ne changent pas d'ordre, et il change silencieusement le jour où elles en changent.

Encore faut-il savoir de quelle stabilité on parle. Pritchard et Wicenec (2024) fournissent le vocabulaire permettant de raisonner rigoureusement sur ces garanties, en distinguant la simple relance, la répétition, le recalcul, la reproduction et la réplication. Leur distinction entre réplication au bit près et équivalence utile nous est directement opératoire, car elle oblige à définir le niveau d'équivalence visé avant de prétendre le vérifier. Appliquée à une migration, elle impose de choisir : exige-t-on des sorties identiques au bit près, ou l'égalité des seuls indicateurs publiés ? La réponse détermine le protocole de validation, et elle n'est pas technique mais métier.

La question des garanties déborde le seul déterminisme, car elle porte aussi sur le comportement en cas d'anomalie. Les travaux consacrés à la qualité des données, que nous examinons plus loin, s'intéressent à la détection des valeurs invalides, non à ce que le moteur en fait une fois détectées, c'est-à-dire interrompre ou propager. Cette granularité de la défaillance conditionne pourtant la disponibilité du système en exploitation. La détection est un problème étudié, la reprise beaucoup moins.

Nous retenons de cet axe que la littérature pertinente ne provient pas du domaine des bases de données mais du génie logiciel et de la science reproductible. Ce déplacement disciplinaire n'est pas anodin : il suggère que le non-déterminisme des pipelines décisionnels est un angle mort, traité comme un défaut d'implémentation là où il relève d'une propriété à spécifier.

3.4. Troisième axe : établir la correspondance entre deux pipelines

Avant de traduire, il faut comprendre. Lorsque le pipeline d'origine n'a d'autre spécification que lui-même, la reconstruction de sa sémantique devient un problème à part entière. Notre cas est instructif à cet égard : le pipeline a été construit et maintenu au sein de l'entreprise, et nous en sommes le mainteneur. Cette proximité ne supprime pas la difficulté, elle la déplace. Passé quelques dizaines de nœuds, un graphe cesse d'être tenu en mémoire par qui que ce soit, y compris par celui qui l'a écrit, et reconstruire sa logique d'ensemble nous a effectivement demandé l'analyse systématique de son format de sérialisation. Il s'agit donc d'une rétro-ingénierie menée par le mainteneur sur son propre travail. Trois stratégies coexistent dans la littérature, et elles se distinguent moins par leurs résultats que par ce qu'elles exigent en entrée.

La première repose sur une taxonomie d'opérateurs. Vassiliadis et al. (2009) classent les activités ETL selon leur cardinalité en schéma et en tuples, et en dégagent une forme normale. L'intérêt pour une migration est direct : disposer d'une classification indépendante de l'outil permet d'établir méthodiquement la correspondance entre un nœud du pipeline source, une étape du pipeline cible et un opérateur relationnel, et surtout d'identifier les constructions qui n'ont d'équivalent ni dans l'un ni dans l'autre. C'est la stratégie la moins outillée, mais la seule qui ne suppose aucun accès au code de l'outil d'origine.

La deuxième passe par l'extraction du lignage. Cui et Widom (2001) en posent les fondements pour l'entrepôt de données, y compris lorsque la transformation est opaque, c'est-à-dire exactement la situation d'un nœud dont l'implémentation appartient à l'outil. Leur apport est de montrer qu'un lignage

utile peut être reconstruit à partir des seuls schémas d'entrée et de sortie, au prix d'une précision moindre. Cette voie suppose toutefois que le pipeline s'exprime dans un langage analysable, ce qu'un graphe de nœuds n'est qu'après interprétation de son format de sérialisation.

La troisième stratégie est la traduction proprement dite. Böhm et al. (2026) en donnent une déclinaison industrielle récente, structurée en quatre phases (analyse, enrichissement, synthèse, transition) organisées autour d'un modèle pivot indépendant des technologies d'origine et de destination. Leur apport principal n'est pas l'automatisation de la traduction, souvent illusoire, mais ce modèle intermédiaire, qui rend explicite une logique métier jusque-là dissoute dans l'outil.

Aucune de ces trois stratégies ne s'applique telle quelle à notre problème. La taxonomie ne dit rien de l'ordre des opérations ni de leurs paramètres ; le lignage suppose un langage analysable ; la traduction automatique suppose des règles préexistantes pour le couple de langages considéré, et il n'en existe aucune. Le patron de Böhm et al., en revanche, se transpose : il consiste précisément à ne pas traduire directement, mais à reconstruire d'abord une représentation intermédiaire de la logique métier, puis à engendrer la cible à partir de celle-ci.

3.5. Quatrième axe : prouver que la migration n'a rien changé

Les approches de validation se laissent ordonner selon la force de la garantie qu'elles procurent et, symétriquement, selon leur coût de mise en œuvre.

Au sommet se trouve la preuve formelle d'équivalence. Zhou et al. (2022) établissent l'équivalence de deux requêtes sous la sémantique des multi-ensembles, c'est-à-dire sur des tables comportant des doublons, ce qui correspond à la situation réelle ; leur évaluation chiffrée montre l'écart considérable de couverture avec les approches antérieures fondées sur la seule sémantique ensembliste. He et al. (2024) proposent une vérification bornée qui admet les contraintes d'intégrité et, surtout, produit un contre-exemple lorsque l'équivalence échoue. Cette propriété est décisive en pratique, car un verdict négatif sans contre-exemple n'oriente aucune correction. Ces travaux partagent cependant une limite que nous devons souligner : ils raisonnent sur des paires de requêtes, non sur des chaînes de traitement complètes.

Au niveau suivant se situe le test différentiel fondé sur un oracle construit. Rigger et Su (2020) engendrent des requêtes dont ils connaissent par construction au moins une ligne du résultat attendu, contournant ainsi l'absence de vérité de terrain. Le principe se transpose à la comparaison de deux implémentations d'un même pipeline, et il éclaire surtout, par contraste, une particularité de notre problème : la chaîne d'origine étant encore exécutable, elle nous fournit elle-même l'oracle, à condition d'en archiver les sorties avant son démantèlement. Cette contrainte de calendrier, rarement discutée dans la littérature, conditionne toute la validation.

Vient ensuite l'assertion déclarative. Schelter et al. (2018) proposent d'exprimer des contraintes de qualité compilées en requêtes d'agrégation, selon une analogie explicite avec les tests unitaires. L'approche reste applicable lorsque aucune référence n'existe, puisqu'elle vérifie des propriétés et non des valeurs. Une migration ne se valide d'ailleurs pas le jour de la bascule : une chaîne périodique doit être surveillée sur plusieurs exécutions, la dérive des données amont produisant des écarts que le portage n'explique pas.

Deux constats ressortent de cet axe. Le premier est que la force de la garantie décroît très vite lorsque l'on passe de la requête au pipeline : il n'existe, à notre connaissance, aucun travail traitant l'équivalence d'une chaîne de traitement complète. Le second est qu'aucun protocole de validation de migration n'est disponible sous une forme évaluable en accès ouvert ; la documentation de fournisseurs abonde, mais ne propose ni méthode reproductible ni critère explicite. C'est ce vide que notre phase de réalisation se propose de combler, par transposition du test différentiel à l'échelle d'une chaîne complète.

4. Conclusion et perspectives

4.1. Ce que la revue établit

Les quatre axes se répondent. Le premier établit que le lieu du calcul est une propriété du modèle d'exécution et non un détail d'implémentation : un graphe de nœuds matérialise chaque résultat intermédiaire sans vue d'ensemble autorisant une réorganisation (Johnston et al., 2004), là où un moteur à évaluation différée construit d'abord une description du calcul, condition de sa délégation (Armbrust et al., 2015). Cette délégation est étudiée depuis Haas et al. (1997) comme une contrainte d'optimisation. Le pipeline que nous observons n'en tire aucun parti, puisque ses vingt lectures rapatrient les tables entières. La migration ne se réduit donc pas à un changement d'outil : elle déplace la frontière entre ce que calcule la base et ce que calcule le moteur de restitution.

Le deuxième axe nous fournit le vocabulaire des garanties, depuis des disciplines inattendues. Shi et al. (2016) nomment le code qui suppose déterministe une spécification qui ne l'est pas ; Pritchard et Wicenec (2024) distinguent relance, répétition, recalcul, reproduction et réplication. La conséquence est pratique : nous devrons décider du niveau d'équivalence visé avant de prétendre le vérifier, et ce choix relève du métier avant de relever de la technique.

Le troisième axe donne la méthode de reconstruction. Des trois stratégies examinées, c'est le patron à modèle pivot de Böhm et al. (2026) qui se transpose : reconstruire d'abord une représentation intermédiaire de la logique métier, puis engendrer la cible à partir d'elle. La taxonomie de Vassiliadis et al. (2009) nous fournit alors la grille de correspondance des opérateurs, et le lignage au sens de Cui et Widom (2001) un critère de complétude, puisque toute colonne produite doit pouvoir être rattachée aux colonnes sources dont elle dérive.

Le quatrième axe hiérarchise les validations selon la force de la garantie obtenue, de la preuve d'équivalence (Zhou et al., 2022 ; He et al., 2024) au test différentiel contre oracle construit (Rigger et Su, 2020), puis à l'assertion déclarative vérifiée à chaque exécution (Schelter et al., 2018). Il nous livre aussi une contrainte que la littérature ne discute guère : dans notre cas, l'oracle est la chaîne d'origine, et il disparaît avec elle.

4.2. Ce que la revue laisse ouvert

Cinq manques ressortent, et ce sont eux qui délimitent la contribution de notre phase suivante.

Aucun travail que nous ayons identifié ne traite la traduction d'un pipeline de flux visuel vers un langage de requêtes fonctionnel. Les travaux de modernisation dirigée par les modèles portent sur des langages de programmation généralistes ; les travaux consacrés aux processus ETL portent sur l'optimisation d'un flux à l'intérieur de son propre outil, non sur son portage vers un autre moteur.

Les vérificateurs d'équivalence raisonnent sur des paires de requêtes, jamais sur une chaîne complète enchaînant lectures, jointures, agrégations et écritures. La composition des garanties le long d'un pipeline n'est pas traitée : savoir que deux étapes sont équivalentes deux à deux ne dit rien de l'équivalence de leur composition en présence de valeurs manquantes ou de doublons.

Aucun protocole de validation de migration n'est disponible sous une forme évaluable en accès ouvert. La documentation d'éditeurs propose des listes de contrôle, non une méthode reproductible assortie d'un critère de réussite explicite et d'un seuil de tolérance annoncé.

Le mécanisme de délégation des outils décisionnels n'a pas de traitement académique propre : désigné par un terme commercial et documenté par son éditeur, il doit être rattaché à l'optimisation fondée sur les capacités de la source pour être discuté rigoureusement. Il en va de même de la sémantique des deux langages en présence, dont aucune description formelle n'est publiée.

Enfin, la littérature sur la qualité des données traite abondamment la détection des valeurs invalides, mais fort peu ce que le moteur en fait une fois détectées, c'est-à-dire interrompre le traitement ou propager l'erreur comme une valeur. C'est pourtant cette granularité de la défaillance qui décide de la disponibilité du système en exploitation.

4.3. Démarche envisagée pour la phase de réalisation

Nous pouvons désormais reformuler la question posée en introduction en trois sous-questions évaluables, auxquelles notre phase de réalisation devra répondre :

> (i) Quelle représentation intermédiaire permet d'exprimer la logique d'un pipeline de flux indépendamment de l'outil qui l'exécute, et se révèle suffisante pour engendrer le pipeline cible ?
>
> (ii) À quel niveau d'équivalence (égalité au bit près, égalité des multi-ensembles à tolérance numérique annoncée, égalité des seuls indicateurs publiés) la migration doit-elle être validée, et quel protocole l'établit à budget de vérification fini ?
>
> (iii) Que gagne-t-on et que perd-on, mesurément, à supprimer l'étage intermédiaire : durée d'actualisation, volume transféré, nombre d'artefacts à maintenir, comportement en présence d'une donnée invalide ?

Notre démarche se déroulera en cinq temps. Le premier est conservatoire et doit être engagé sans délai : figer l'oracle, c'est-à-dire archiver un export daté des douze tables de sortie et relever les caractéristiques d'exécution de la chaîne actuelle (chronométrage par étape, volume intermédiaire, pic mémoire, configuration) tant qu'elle s'exécute encore. Le deuxième est la reconstruction de la sémantique du pipeline à partir de son format de sérialisation, sous la forme du modèle pivot évoqué plus haut. Le troisième est l'écriture du pipeline cible à partir de ce modèle, en cherchant à maximiser la part du calcul déléguée à la source et en relevant explicitement le point où la délégation se rompt. Le quatrième est la validation : comparaison table à table et ligne à ligne contre l'oracle, avec un seuil de tolérance déclaré avant l'expérience, puis expression des invariants métier sous forme d'assertions déclaratives évaluées à chaque actualisation. Le cinquième est la mesure comparée. Une première observation nous en fixe l'ordre de grandeur, sans valeur de résultat tant qu'un protocole ne l'encadre pas : la chaîne actuelle s'exécute en moins de cinq minutes, et l'actualisation d'une version reconstruite, réalisée à titre exploratoire, en moins de deux. Si ce gain se confirme, il restera modeste en durée, et l'intérêt de la migration devra se chercher ailleurs.

Notre critère de réussite est binaire et vérifiable : les douze tables produites par le nouveau pipeline doivent coïncider avec l'oracle, ligne à ligne, à la tolérance annoncée près. Tout écart résiduel devra être expliqué, ou bien assumé comme une correction de la chaîne d'origine, ce qui suppose alors de le documenter comme tel. Cette exigence est plus sévère qu'il n'y paraît, puisqu'elle nous interdit d'attribuer un écart à la migration après coup.

Nous identifions trois risques. Le premier, irréversible, est la perte de l'oracle si la chaîne d'origine était démantelée avant archivage. Le deuxième est l'existence d'une opération sans équivalent dans le langage cible, qui obligerait à conserver un traitement résiduel hors du moteur de restitution et rendrait la suppression de l'étage intermédiaire partielle plutôt que complète. Le troisième est la dérive des données amont pendant la période de comparaison, qui produit des écarts que la migration n'explique pas : elle nous impose de comparer d'abord sur un jeu figé, puis seulement en exploitation.

Ainsi posée, la contribution que nous attendons de la phase de réalisation n'est pas la migration elle-même, qui relève de l'ingénierie, mais ce qui la rend vérifiable : une représentation intermédiaire de la logique métier indépendante des deux outils, un protocole transposant le test différentiel de l'échelle de la requête à celle de la chaîne complète, et une mesure documentée de ce que le déplacement du calcul change effectivement.

Références

Quinze références, retenues sur un corpus initial d'une quarantaine (voir section 3.1), toutes librement accessibles en ligne. Les documentations d'éditeurs et les outils logiciels sont listés séparément en annexe, afin de ne pas les confondre avec des travaux évalués par les pairs.

Armbrust, M. et al. (2015). *Spark SQL: Relational Data Processing in Spark*. SIGMOD '15.
https://people.csail.mit.edu/matei/papers/2015/sigmod_spark_sql.pdf

Böhm, T. et al. (2026). *Model-Driven Legacy System Modernization at Scale*. ReCode '26 @ ICSE · arXiv:2602.04341. https://arxiv.org/abs/2602.04341

Cui, Y., Widom, J. (2001). *Lineage Tracing for General Data Warehouse Transformations*. VLDB '01. https://www.vldb.org/conf/2001/P471.pdf

Haas, L. M., Kossmann, D., Wimmers, E. L., Yang, J. (1997). *Optimizing Queries across Diverse Data Sources*. VLDB '97, 276–285. https://www.vldb.org/conf/1997/P276.PDF

He, Y., Zhao, P., Wang, X., Wang, Y. (2024). *VeriEQL: Bounded Equivalence Verification for Complex SQL Queries with Integrity Constraints*. OOPSLA 2024 · arXiv:2403.03193. https://arxiv.org/abs/2403.03193

Johnston, W. M., Hanna, J. R. P., Millar, R. J. (2004). *Advances in Dataflow Programming Languages*. ACM Computing Surveys 36(1), 1–34. https://www.cs.tufts.edu/comp/150CMP/papers/dataflow-johnston-acm04.pdf

Pritchard, N. J., Wicenec, A. (2024). *Formal Definition and Implementation of Reproducibility Tenets for Computational Workflows*. Préprint arXiv:2406.01146. https://arxiv.org/abs/2406.01146

Rigger, M., Su, Z. (2020). *Testing Database Engines via Pivoted Query Synthesis*. USENIX OSDI 2020 · arXiv:2001.04174. https://arxiv.org/abs/2001.04174

Schelter, S., Lange, D., Schmidt, P., Celikel, M., Biessmann, F., Grafberger, A. (2018). *Automating Large-Scale Data Quality Verification*. PVLDB 11(12), 1781–1794. http://www.vldb.org/pvldb/vol11/p1781-schelter.pdf

Sculley, D. et al. (2015). *Hidden Technical Debt in Machine Learning Systems*. NeurIPS 2015. https://proceedings.neurips.cc/paper_files/paper/2015/file/86df7dcfd896fcaf2674f757a2463eba-Paper.pdf

Shi, A., Gyori, A., Legunsen, O., Marinov, D. (2016). *Detecting Assumptions on Deterministic Implementations of Non-deterministic Specifications*. IEEE ICST 2016 (outil NonDex). https://www.cs.cornell.edu/~legunsen/pubs/ShiETAL16NonDex.pdf

Simitsis, A., Vassiliadis, P., Sellis, T. (2005). *Optimizing ETL Processes in Data Warehouses*. ICDE 2005, 564–575. https://www.cs.ubc.ca/~rap/teaching/534P/2011/readings/etl.pdf

Vassiliadis, P., Simitsis, A., Baikousi, E. (2009). *A Taxonomy of ETL Activities*. DOLAP '09. https://www.cs.uoi.gr/~pvassil/publications/2009_DOLAP_ETL/DOLAP_2009_ETL.pdf

Yan, C., Lin, Y., He, Y. (2023). *Predicate Pushdown for Data Science Pipelines*. PACMMOD 1(2) · SIGMOD 2023. https://www.microsoft.com/en-us/research/wp-content/uploads/2023/05/predicate_pushdown_final.pdf

Zhou, Q., Arulraj, J., Navathe, S., Harris, W., Wu, J. (2022). *SPES: A Symbolic Approach to Proving Query Equivalence Under Bag Semantics*. ICDE 2022 · arXiv:2004.00481. https://arxiv.org/abs/2004.00481

Annexe A — Rattachement des références aux axes de la revue

Axe 1 : où et quand le calcul s'exécute · Axe 2 : ce que le modèle d'exécution garantit · Axe 3 : établir la correspondance entre deux pipelines · Axe 4 : prouver que la migration n'a rien changé. Un point marque l'axe auquel la référence contribue ; la dernière colonne indique les sections où elle est discutée.

Référence	Axe 1	Axe 2	Axe 3	Axe 4	Sections
Armbrust et al. (2015)	●				3.2, 4.1
Böhm et al. (2026)			●		3.4, 4.1
Cui et Widom (2001)			●		3.4, 4.1
Haas et al. (1997)	●				3.2, 4.1
He et al. (2024)				●	3.5, 4.1
Johnston et al. (2004)	●				3.2, 4.1
Pritchard et Wicenec (2024)		●		●	3.3, 4.1
Rigger et Su (2020)				●	3.5, 4.1
Schelter et al. (2018)				●	3.5, 4.1, 4.3
Sculley et al. (2015)					3.1 (motivation)
Shi et al. (2016)		●			3.3, 4.1
Simitsis et al. (2005)	●			●	3.2
Vassiliadis et al. (2009)			●		3.4, 4.1
Yan et al. (2023)	●			●	3.2
Zhou et al. (2022)				●	3.5, 4.1

Table 1 — Contribution de chacune des quinze références aux quatre axes transverses.

Annexe B — Documentations d'éditeurs et outils logiciels

Sources techniques et outils mobilisés dans le texte ou envisagés pour la phase de réalisation ; non évalués par les pairs, ils sont tenus à l'écart des références.

Microsoft Learn — Query folding (Power Query). Définition par l'éditeur du mécanisme de délégation, des étapes délégables et du point de rupture. Sections 2.3 et 3.2. https://learn.microsoft.com/fr-fr/power-query/query-folding-basics

Microsoft Learn — Spécification du langage M. Description de référence du modèle d'exécution du langage de transformation cible : fonctionnel, à valeurs immuables, à évaluation paresseuse. Section 2.2. https://learn.microsoft.com/fr-fr/powerquery-m/power-query-m-language-specification

KNIME — Database Extension Guide. Documentation des nœuds qui composent du SQL sans matérialiser. Elle établit que l'outil de flux sait déléguer, et que le pipeline étudié n'en fait pas usage. Sections 2.2 et 3.2. https://docs.knime.com/ap/latest/databases/

Deequ (AWS Labs). Implémentation de référence de Schelter et al. (2018) : forme concrète d'une contrainte de qualité déclarative compilée en agrégations. Sections 3.5 et 4.3. https://github.com/awslabs/deequ

Great Expectations. Assertions déclaratives sur les données, documentation de validation engendrée. Candidat pour les invariants métier, section 4.3. https://docs.greatexpectations.io/

data-diff (Datafold, projet archivé). Comparaison ligne à ligne entre bases hétérogènes ; référence outillée de la réconciliation, projet non maintenu. Section 4.3. https://github.com/datafold/data-diff

Cosette (démonstrateur). Prouveur d'équivalence de requêtes SQL utilisable en ligne ; illustre concrètement ce que « prouver » signifie dans la hiérarchie de la section 3.5. https://cosette.cs.washington.edu